

해커톤 협업 강의

개발자와 비개발자가 AI와 함께 프로젝트를 완성하는 방법

전시진 (SIREAL) · 성균관대학교

오늘의 흐름

01 소통 창구 만들기

02 아이디어션

03 아이디어 선별

04 문서화

05 구현

06 오류수정

07 발표준비

08 자주 묻는 질문

아이디어션 ↔ 아이디어 선별은 반복 루프입니다.

이 강의는 무엇을 위한 것인가

- 개발자 + 비개발자가 섞인 팀이
- AI와 함께 해커톤 프로젝트를 끝까지 완성이 하기 위한 협업 가이드
- 직접 코딩·만드는 방법이 아니라 AI에게 잘 시키는 방법

기억할 핵심 두 가지

1. 이제는 직접 만드는 게 아니라 AI에게 "잘 시키는" 시대
 - 잘 시키려면 명확한 기록과 문서가 필요하다
2. 모든 대화·자료·결정은 노선에 기록으로 남긴다
 - 그 기록이 곧 AI가 일하는 재료(컨텍스트)

전체 흐름

소통 창구 → 아이디어션 ↔ 아이디어 선별 → 문서화 → 구현 → 오류수정 → 발표준비

- 아이디어션 ↔ 아이디어 선별 = 반복 루프
- 괜찮은 아이디어 → 선별·검증 → 결과 보고 다시 아이디어 발전

각 단계마다 같은 5요소

방법·순서 → 도구 → 규칙 → 실습 → 점검

- 오늘 1~7번 섹션 모두 이 순서로 설명한다
- 팀 활동도 이 순서로 진행하면 된다

01

소통 창구 만들기

1. 소통 창구 — 방법·순서

1. 노션 페이지 생성
2. 팀원 초대
3. 페이지 안에 데이터베이스 생성
4. 각종 문서를 종류별로 정리

1. 소통 창구 — 도구

- **Notion** (나중에 MCP 연결)
- 복잡한 DB 구조는 필요 없다
- **종류** 속성(select) 하나면 충분
 - 회의록 / 자료 / 할 일 / 문서 등

1. 소통 창구 — 규칙

1. 매 회의 시작 시 AI 회의 노트를 켜다 ← 가장 중요
2. 모든 자료·기록은 노션 한곳에
3. 제목은 알아보기 쉽게
4. DB에는 **종류** 속성 하나만
5. 좋은 의견·결정은 반드시 문서로 남긴다

⚠ AI 노트를 켜지 않은 회의 = 흔적이 사라진다

검

실습

- 노션 페이지 만들고 팀원 초대
- DB 생성 → 첫 항목(회의록) 올려보기

점검

- 전원 초대됐는가?
- 문서가 한곳에 종류별로 쌓이는가?

02

2. 아이디어션 — 방법·순서

1. 문제 정의
2. 자료 수집
3. 회의로 아이디어 쏟아내기

2. 아이디어션 — 도구

- 노션 AI 노트 (우선)
- 클로바노트 (대안 — 웹 접속 가능)
- AI 음성 모드, 검색

팀원 전체가 노션 AI 노트를 쓸 수 없으면 클로바노트로 대체

2. 아이디어션 — 규칙

- 회의 시작과 동시에 AI 노트부터
- 다른 사람 아이디어 비판 금지
- 긍정·확장만 — "더 이렇게 하면?"
- 판단 보류, 양으로 승부
- 떠오르면 즉시 기록

점검

실습

- 클로바노트로 아이디어 회의 → 문서로 정리
- (또는) 노션 AI 노트로 동일 실습

점검

- 정리된 문서 공유
- 흥미도·실현성 코멘트

03

3. 아이디어 선별 — 방법·순서

1. 후보 나열
2. 후보별 딥리서치 → 근거 수집
3. 심사 기준 적용 → 점수
4. 우선순위 → 1개 확정

딥리서치는 시간이 걸린다 → 후보 정해지는 대로 바로 돌려두기

3. 심사 기준 5가지

1. 문제 정의 및 사회적 필요성
2. 창의성 및 독창성
3. AI 활용의 적절성 및 기술 타당성
4. 실현 가능성 및 완성도
5. 기대 효과 및 파급력

3. 아이디어 선별 — 도구

도구	특징
Gemini Deep Research	긴 리포트, 출처 링크
ChatGPT Deep Research	단계별 리서치, 근거 풍부
Claude	긴 문서 분석·구조화 평가
Perplexity	빠른 팩트체크

- 평가표: Notion / 스프레드시트
- 예시 프롬프트: `Example_prompt/` 폴더

3. 아이디어 선별 — 규칙

- 심사위원 관점·기준으로 본다
- "이거 좋은데요?" 반응 = 신호
- 딥리서치는 후보 확정 즉시 실행
- 결과 + 점수를 노선에 함께 기록
- 선정 이유를 문서에 남긴다

• 심심

실습

- 아이디어 1개 → 딥리서치 프롬프트 작성
- 심사 기준표에 대입 → 점수화 → 1개 선택

점검

- 선정 사유 발표
- 기준이 명확한가?

04

4. 문서화 — 방법·순서

1. 문서 목록 잡기
2. 마크다운 작성
3. 검토
4. docs/ 폴더에 묶기
5. 지속 업데이트

4. 문서 목록 (예시)

- 필수 — PRD, TRD, README
- 권장 — 디자인 가이드, 화면·기능 정의, Changelog
- 선택 — 데이터 모델, API 명세, 컨벤션, 배포 가이드, 발표 개요

모든 문서 → 프로젝트 루트의 docs/ 폴더

4. 문서화 — 도구 · 규칙

도구 — 마크다운 에디터, Claude, Cursor

- AI에 너무 의존 → 문장만 늘어난다
- 마크다운 = AI가 가장 좋아하는 형식
- 2,000줄 이내 — 짧을수록 좋음
- 길어지면 주제별로 쪼개기

- AI는 결정을 대신하지 않는다
- 프롬프트 입력자의 기반에 맞춰 결과를 낸다
- 의사결정은 팀이 한다

실습

docs/ 폴더 + PRD 1장 작성

점검

"AI가 읽기 좋은 구조인가?" 상호 리뷰

05

5. 구현 — 방법·순서

1. Repo·배포 준비 (GitHub + Cursor clone)
2. docs/ 폴더 확인
3. 구현 범위 정하기 (PRD + TRD)
4. Agent mode로 구현 (필요 시 Plan mode)
5. 확인 (브라우저)
6. Commit + Push

5. Agent mode가 기본

- Agent mode — docs/ @첨부 → "○○ 기능 구현해줘"
- Plan mode — 범위·방향이 불명확할 때만 추가
- Commit + Push — 팀원 누구나 (개발자 전담 아님)

5. docs/ → 구현 시 활용

문서	역할
PRD	무엇을 — 범위·기능
TRD	어떻게 — 스택·구조
README	프로젝트 맥락
화면·기능 정의	구현 단위 (1개씩)
Changelog	구현·수정 이력

5. 구현 — 잘 시키는 법 (1)

1. 문서를 먼저 붙인다 — @docs/PRD.md
2. 역할을 나눈다 — PRD=범위, TRD=제약
3. 한 번에 하나씩 — 기능·화면 1개 단위
4. 제약을 문서에서 — TRD·디자인 가이드 그대로

5. 기대 결과를 문서로 — acceptance criteria 인용

6. 어긋나면 문서부터 — 코드 vs 문서, 팀 확인

7. 확인 후 Commit + Push

8. 안 되면 다시 — 지시문·문서 수정 후 반복

실습

Repo → PRD @첨부 → Agent → 확인 → Push

점검

docs/와 실제 동작 일치? GitHub 커밋?

6. 오류수정 — 방법·순서

1. 문제 발견·기록 (화면 / 순서 / 결과)
2. 재현 (2~3회)
3. `docs/` 와 대조 (버그? 문서 오류? 기획 변경?)
4. **Agent mode** 수정 (필요 시 Plan)
5. **확인** (브라우저)
6. `docs/Changelog.md` 기록
7. **Commit + Push**

6. 비개발자도 오류수정을 한다

역할	하는 일
비개발자 (주도)	발견·재현·지시·확인·Changelog·Push
개발자 (보조)	지시 보완·에러 로그·배포 점검

비개발자 = 발견 · 재현 · 기록 · 확인 · Commit + Push

6. 오류 수정 지시문 (1)

1. 증상을 구체적으로 — "로그인 클릭 → 흰 화면"
2. 기대 결과는 docs/에서 — PRD 기준 명시
3. 재현 순서 그대로 — 1, 2, 3단계
4. 한 번에 하나만 — 오류 1건 = 지시 1개

0. 소위 두영지서판 (2)

5. 스크린샷·녹화 첨부
6. 직접 확인 — AI 말만 믿지 않는다
7. 확인 후 Commit + Push — 개발자에게만 말기지 않는다
8. 고치기 ≠ 기능 추가 — 추가는 New 세션 + 5번 흐름

실습

오류 재현 → Agent 수정 → Changelog → Push

07

7. 발표준비 — 슬라이드 3단계

1. 문서 정리·목차 구성

- PRD, Changelog, README → 핵심 메시지·목차 확정

2. 세부 내용 작성

- 확정 목차별 문장·수치·시연 포인트

3. 디자인 적용

- 내용 + 디자인 가이드 → 슬라이드 완성

목차·내용 먼저, 디자인은 마지막

7. docs/ → 발표 · 도구

문서	발표에서
PRD	문제·기능·기대 효과
Changelog	"우리가 만든 것"
README	프로젝트 소개
디자인 가이드	슬라이드 톤·색

도구 — Gemini, NotebookLM, Claude, Canva, Gamma 등

7. 발표 준비 - 규칙 & 실습

- docs/ 에서 시작 (새로 쓰지 않는다)
- 목차 먼저 확정
- 3단계 순서 지키기
- AI 결과 ↔ docs/ 대조
- 시연 리허설 + 시간 맞추기

실습

PRD·Changelog @첨부 → 목차 5~7장 → 내용 → 디자인 1장

FAQ

자주 묻는 질문

전시진 (SIREAL) · 성균관대학교 해커톤 협업 강의

Q. 소통 창구 만들 시간이 어딴어요?

바쁠수록 더 필요하다.

- "아까 누가 뭐 말했더라" 반복 = 시간 낭비
- 노선 페이지 + 초대 = 5분
- 이후 문서화 → 구현 → 발표까지 전부 연결

Q. 비개발자는 뭘 하나요?

할 일이 더 많아진다.

- 문제 정의 · 딥리서치 · 문서화(PRD)
- 지시문 작성 · 오류 수정 (Agent)
- Commit + Push · 발표 준비
- "무엇을 왜 만드는가" = 비개발자 강점

Q. AI 결과를 그대로 믿어도 되나요?

안 된다.

- AI는 결정을 대신하지 않는다
- 코드 → 직접 실행·테스트
- 문서·리서치 → 출처·사실 확인
- 딥리서치 통계 → 교차 확인

Q. AI를 한 번도 안 써봤어요

AI 노트 켜기부터.

1. 회의 때 노션 AI 노트 / 클로바노트
2. 말하면 기록 남음 = 가장 쉬운 시작
3. 익숙해지면 → 검색 → 딥리서치 → 지시문
4. 역할 나눠서 먼저 시도하는 사람부터

Q. 비개발자도 Commit + Push?

한다. 해커톤 팀원 누구나.

- 문제 재현 → docs/ 기준 Agent 지시
- 브라우저 확인 → Changelog → Push
- 개발자만의 일 아님

정리

소통 창구 → 아이디어션  선별 → docs/ → Agent 구현 → 오류수정 → 발표

기록하고, 문서화하고, AI에게 잘 시키자.

질문 · 실습 시간